



TRANSPORT AND
TELECOMMUNICATION
INSTITUTE

ENGINEERING FACULTY

Laboratory work N2

Course: **Programming 2**

Theme: **Structures, files and menu driven data processing**

Student: Igors Oļeņikovs

Student code: 93642

Group: 4501BTA

CONTENTS

1.	Laboratory work task	3
2.	Individual task.....	3
3.	Algorithm data flow	4
4.	Algorithm description	5
5.	Source code	7
6.	Running program example.....	11
7.	Testing.....	12
8.	Conclusions	14

1. LABORATORY WORK TASK

Create an algorithm according to individual task. User should be able to:

- **save user entered data (stored in structure) to file;**
- **view file data (read data from file and print to the screen) in a clear way;**
- **perform additional operation according to the individual task.**

Create a console application in C/C++ based on created algorithm where:

- **data structure according to the individual task is created and used;**
- **if data are stored in array then only native arrays are used (vectors, lists etc. are not allowed here);**
- **user-defined functions are created and being used** (at least one function for any subtask);
- **files are being used** (to store data);
- **menu is available** (should be at least 4 menu items: add data, view data, ind.task, exit);

2. INDIVIDUAL TASK

Individual task number is calculated as $93642 \% 20 = 2$.

Task 2: Structure fields: shop title, address, tel.number. Operation: define tel.number for shop with specified address.

3. ALGORITHM DATA FLOW

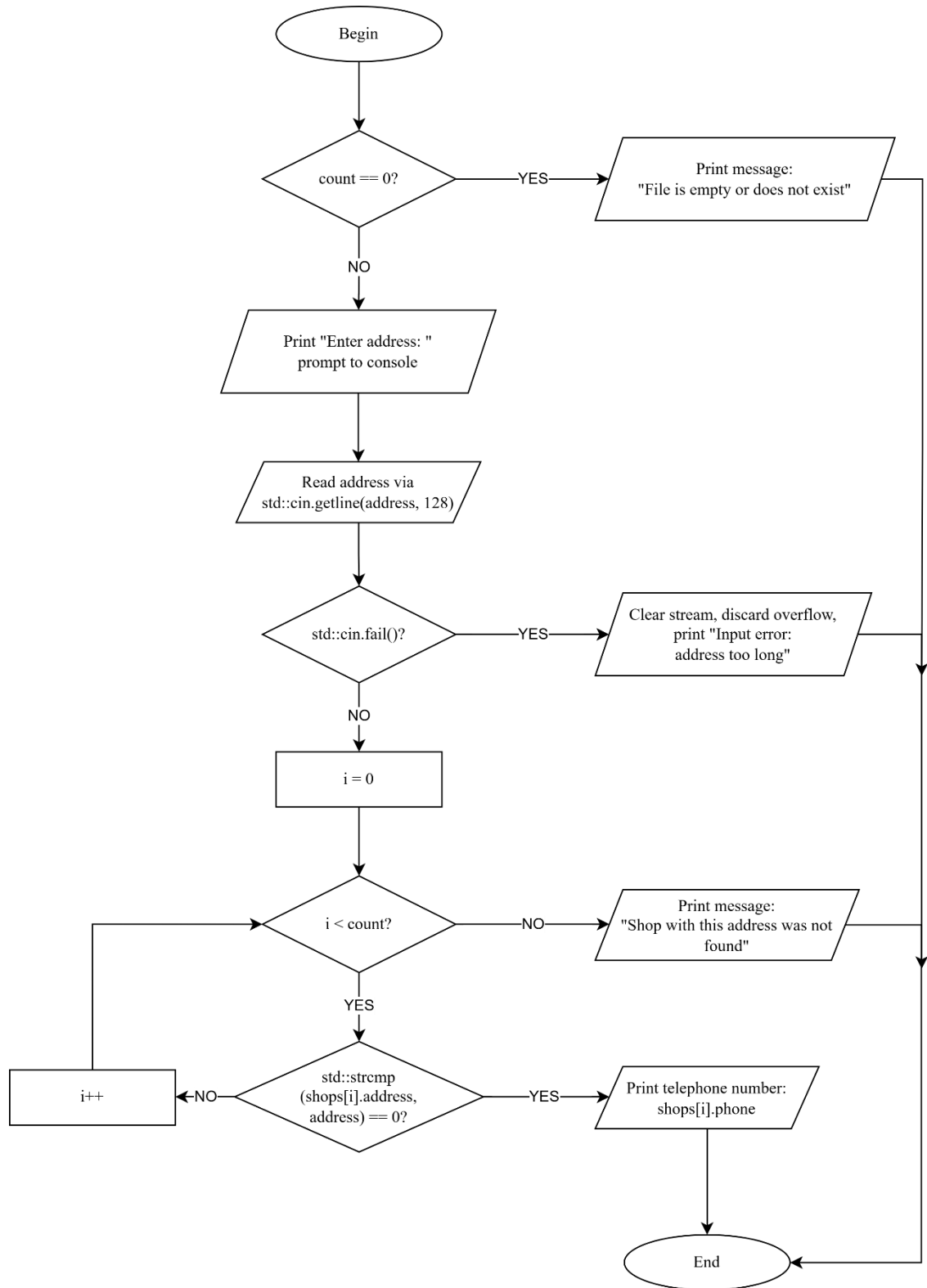


Fig. 1. `definePhoneByAddress` function

4. ALGORITHM DESCRIPTION

The program manages a shop database using a fixed size array of Shop structures (`MAX_SHOPS = 100`). Data is persisted in the binary file `shops.dat`: the first 4 bytes store the record count as `std::int32_t`, followed by many Shop records written as raw bytes.

Structure: Shop

```
struct Shop
{
    char title[64];
    char address[128];
    char phone[32];
};
```

Trivially copyable - contains only plain char arrays, which allows raw binary I/O via `reinterpret_cast`.

Why `std::int32_t` for the count header

The file header uses `std::int32_t` (from `<cstdint>`) instead of plain `int` or `std::size_t`. Plain `int` has a platform-dependent size (the C++ standard only guarantees at least 16 bits), and `std::size_t` varies between 4 and 8 bytes across architectures. `std::int32_t` is guaranteed to be exactly 32 bits on all platforms, ensuring the binary file format is consistent. Both **`readShops`** and **`writeShops`** use `std::int32_t` for the count header, keeping the read and write sides in agreement.

Input validation (stdin checks)

The program uses `std::cin.getline()` for all string input. After each `getline` call, `std::cin.fail()` is checked to detect buffer overflow (user typed more characters than the buffer can hold). On overflow:

- `std::cin.clear()` resets the stream fail state.
- `std::cin.ignore` discards the leftover input.
- A field-specific error message is printed.
- The operation is aborted without saving.

This pattern is used in:

- **`addData()`** for title, address, and phone fields.
- **`definePhoneByAddress()`** for the search address.
- **`main()`** for the menu choice (`std::cin >> menuItem` with `fail()` check for non-numeric input).

Functions

readShops(Shop *shops)

Opens shops.dat in binary input mode, reads the 4-byte std::int32_t count, validates it (non-negative and <= MAX_SHOPS), then reads count Shop records into the buffer. Returns the number of shops read or 0 on any error (does not distinguish between an empty file and an I/O error).

writeShops(const Shop *shops, std::size_t count)

Opens shops.dat in binary output mode (overwriting previous content), writes the count as a 4-byte std::int32_t, then writes all count Shop records as raw bytes. Returns true on success, false on failure. On failure the file may be left partially written, there is no atomic-write mechanism.

addData(Shop *shops, std::size_t *count)

Prompts for title, address and phone via std::cin.getline() with overflow detection on each field. Stores the entry at index *count, increments the count, and persists via writeShops(). Rolls back *count if the write fails. Rejects when count >= MAX_SHOPS.

viewData(const Shop *shops, std::size_t count)

Iterates over the array and prints all records with a 1-based index. Prints a message if count is 0.

definePhoneByAddress(const Shop *shops, std::size_t count)

Prompts for an address with overflow detection, then performs a case sensitive linear search using std::strcmp. Prints the phone number of the first match or a "not found" message.

main()

Declares the shops array, loads data via readShops(), then runs an infinite menu loop (1-Add, 2-View, 3-Search, 4-Exit). Non-numeric menu input is caught via std::cin.fail() and the stream is cleaned. The loop exits on option 4.

5. SOURCE CODE

```
#include <cstdint>
#include <cstring>
#include <cstddef>
#include <fstream>
#include <iostream>
#include <limits>

const std::size_t MAX_SHOPS = 100;
const char *DATABASE_FILE = "shops.dat";

struct Shop
{
    char title[64];
    char address[128];
    char phone[32];
};

std::size_t readShops(Shop *shops)
{
    std::ifstream file(DATABASE_FILE, std::ios::binary);
    std::int32_t rawCount = 0;
    std::size_t count;

    if (!file) return (0);
    file.read(reinterpret_cast<char *>(&rawCount), sizeof(rawCount));
    if (!file || rawCount < 0
        || static_cast<std::size_t>(rawCount) > MAX_SHOPS)
        return (0);
    count = static_cast<std::size_t>(rawCount);
    if (count > 0)
    {
        file.read(reinterpret_cast<char *>(shops), sizeof(Shop) * count);
        if (!file) return (0);
    }
    return (count);
}

bool writeShops(const Shop *shops, std::size_t count)
{
    std::ofstream file(DATABASE_FILE, std::ios::binary);
    std::int32_t rawCount = static_cast<std::int32_t>(count);

    if (!file)
    {
        std::cout << "Cannot open file for writing\n";
    }
}
```

```

        return (false);
    }
    file.write(reinterpret_cast<const char *>(&rawCount), sizeof(rawCount));
    file.write(reinterpret_cast<const char *>(shops), sizeof(Shop) * count);
    if (!file)
    {
        std::cout << "Cannot write data to file\n";
        return (false);
    }
    return (true);
}

void addData(Shop *shops, std::size_t *count)
{
    std::size_t i = *count;

    if (i >= MAX_SHOPS)
    {
        std::cout << "Database is full\n";
        return;
    }

    std::memset(&shops[i], 0, sizeof(Shop));

    std::cout << "Enter shop title: ";
    std::cin.getline(shops[i].title, 64);
    if (std::cin.fail())
    {
        std::cin.clear();
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
        std::cout << "Input error: title too long\n";
        return;
    }
    std::cout << "Enter address: ";
    std::cin.getline(shops[i].address, 128);
    if (std::cin.fail())
    {
        std::cin.clear();
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
        std::cout << "Input error: address too long\n";
        return;
    }
    std::cout << "Enter tel. number: ";
    std::cin.getline(shops[i].phone, 32);
    if (std::cin.fail())
    {
        std::cin.clear();

```

```

        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
        std::cout << "Input error: phone number too long\n";
        return;
    }
    *count = i + 1;
    if (!writeShops(shops, *count))
    {
        *count = i;
        return;
    }
    std::cout << "Data saved\n";
}

void viewData(const Shop *shops, std::size_t count)
{
    if (!count)
    {
        std::cout << "File is empty or does not exist\n";
        return;
    }
    std::cout << "\nShops in file: " << count << '\n';
    for (std::size_t i = 0; i < count; i++)
    {
        std::cout << "Shop " << i + 1 << ":\n"
            << "  Title: " << shops[i].title << '\n'
            << "  Address: " << shops[i].address << '\n'
            << "  Tel. number: " << shops[i].phone << '\n';
    }
}

void definePhoneByAddress(const Shop *shops, std::size_t count)
{
    char address[128];

    if (!count)
    {
        std::cout << "File is empty or does not exist\n";
        return;
    }
    std::cout << "Enter address: ";
    std::cin.getline(address, 128);
    if (std::cin.fail())
    {
        std::cin.clear();
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
        std::cout << "Input error: address too long\n";
        return;
    }
}

```

```

    }
    for (std::size_t i = 0; i < count; i++)
    {
        if (std::strcmp(shops[i].address, address) == 0)
        {
            std::cout << "Tel. number: " << shops[i].phone << '\n';
            return;
        }
    }
    std::cout << "Shop with this address was not found\n";
}

int main(void)
{
    Shop shops[MAX_SHOPS];
    std::size_t count = readShops(shops);
    int menuItem;

    std::cout << "Task 2\n";
    for (;;)
    {
        std::cout << "\nMenu:\n"
            << "1. Add data\n"
            << "2. View data\n"
            << "3. Define tel. number by address\n"
            << "4. Exit\n"
            << "Enter menu item: ";
        if (!(std::cin >> menuItem))
        {
            std::cin.clear();
            std::cout << "Invalid menu item\n";
        }
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
        if (menuItem == 1)
            addData(shops, &count);
        else if (menuItem == 2)
            viewData(shops, count);
        else if (menuItem == 3)
            definePhoneByAddress(shops, count);
        else if (menuItem == 4)
            break;
        else
            std::cout << "Invalid menu item\n";
    }
    return (0);
}

```

6. RUNNING PROGRAM EXAMPLE

```
Terminal - s@Lin: ~/TSI/Programming/LAB2_2
File Edit View Terminal Tabs Help
s@Lin:~/TSI/Programming/LAB2_2$
s@Lin:~/TSI/Programming/LAB2_2$ make
g++ -Wextra -Wall -Werror -pedantic -std=c++17 -c -o main.o main.cpp
g++ -Wextra -Wall -Werror -pedantic -std=c++17 -o shops main.o
rm -f main.o
s@Lin:~/TSI/Programming/LAB2_2$ ./shops
Task 2

Menu:
1. Add data
2. View data
3. Define tel. number by address
4. Exit
Enter menu item: 1
Enter shop title: Tets
Enter address: Valnu 3
Enter tel. number: 22365214
Data saved

Menu:
1. Add data
2. View data
3. Define tel. number by address
4. Exit
Enter menu item: 1
Enter shop title: Ferder
Enter address: Perses 5
Enter tel. number: 25987654
Data saved

Menu:
1. Add data
2. View data
3. Define tel. number by address
4. Exit
Enter menu item: 2

Shops in file: 2
Shop 1:
  Title: Tets
  Address: Valnu 3
  Tel. number: 22365214
Shop 2:
  Title: Ferder
  Address: Perses 5
  Tel. number: 25987654

Menu:
1. Add data
2. View data
3. Define tel. number by address
4. Exit
Enter menu item: 3
Enter address: Perses 5
Tel. number: 25987654

Menu:
1. Add data
2. View data
3. Define tel. number by address
4. Exit
Enter menu item: 3
Enter address: Perses 5
Tel. number: 25987654

Menu:
1. Add data
2. View data
3. Define tel. number by address
4. Exit
Enter menu item: 4
s@Lin:~/TSI/Programming/LAB2_2$
```

Fig. 2. Running program example

7. TESTING

Table 1

Testing table

Test No.	Input or action	Expected result	Actual result	Conclusion
1	Run make test, which starts with a clean build.	The program compiles without warnings or errors.	Build finished successfully.	Pass
2	Choose View data when shops.dat does not exist.	The program reports that the file is empty or does not exist.	Message "File is empty or does not exist" is shown.	Pass
3	Add a shop with title, address, and telephone number.	The record is saved to the binary file shops.dat.	Message "Data saved" is shown.	Pass
4	Choose View data after adding a shop.	The saved record is printed with all structure fields.	The program prints the saved shop title, address and tel. number.	Pass
5	Search for an address that exists in shops.dat.	The telephone number for that shop is found and printed.	The program prints the expected tel. number.	Pass
6	Search for an address that is not stored in the file.	A not found message is printed.	The program prints "Shop with this address was not found"	Pass
7	Add two shops and choose View data.	Both saved records are printed and the count is 2.	The program prints "Shops in file: 2" and both records.	Pass
8	Add data containing spaces and special characters.	The binary file stores and displays the characters without delimiter problems.	The entered data displayed correctly.	Pass
9	Enter invalid menu values: 99, abc and -1.	The program rejects each invalid value and continues.	The program prints "Invalid menu item"	Pass
10	Choose Exit.	The program terminates normally.	The program exits with status 0.	Pass

Memory leaks – No (Valgrind)

```
Shops in file: 2
Shop 1:
  Title: Tetr
  Address: Brivibas 22
  Tel. number: 23456987
Shop 2:
  Title: Waser
  Address: Perses 44
  Tel. number: 21789456

Menu:
1. Add data
2. View data
3. Define tel. number by address
4. Exit
Enter menu item: 1
Enter shop title: Nemo
Enter address: Lacplesa 2
Enter tel. number: 25741852
Data saved

Menu:
1. Add data
2. View data
3. Define tel. number by address
4. Exit
Enter menu item: 2

Shops in file: 3
Shop 1:
  Title: Tetr
  Address: Brivibas 22
  Tel. number: 23456987
Shop 2:
  Title: Waser
  Address: Perses 44
  Tel. number: 21789456
Shop 3:
  Title: Nemo
  Address: Lacplesa 2
  Tel. number: 25741852

Menu:
1. Add data
2. View data
3. Define tel. number by address
4. Exit
Enter menu item: 4
==104474==
==104474== HEAP SUMMARY:
==104474==    in use at exit: 0 bytes in 0 blocks
==104474==   total heap usage: 7 allocs, 7 frees, 93,104 bytes allocated
==104474==
==104474== All heap blocks were freed -- no leaks are possible
==104474==
==104474== For lists of detected and suppressed errors, rerun with: -s
==104474== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
s@Lin:~/TSI/Programming/LAB2_2$
```

8. CONCLUSIONS

A C/C++ program was developed according to individual task 2. The program uses a structure, native arrays, user defined functions, file operations and a four item menu. All required operations are implemented: data input, file storage, clear data viewing and telephone number search by address.

The testing results confirm that the program works correctly for empty file handling, record insertion, viewing saved records, successful address lookup, unsuccessful address lookup and invalid menu input. The work reinforced practical skills in structure processing, file storage, menu driven console applications and sequential search in native arrays. No algorithmic complications met. The task required approximately 6 hours, primarily spent testing edge cases and report writing.